

HACKER OLMAK YA DA OLMAMAK

[Kaynak](#)

İÇERİK

1. Offansive Security Nedir?
2. Finding Weaknesses: Hedef Sistemde Zafiyet Tespiti ve Keşif
3. Exploiting Weaknesses: Zafiyetleri İstismar Etmek ve Zincirleme Saldırı Çevrimi

Offansive Security Nedir?

Ofansif Güvenlik Paradigması ve Hedef Odaklı Yaklaşım

Offensive Security (Ofansif Güvenlik - sistemleri kırmaya çalışarak zafiyetleri proaktif olarak tespit etme süreci), savunma mimarilerinin aksine, sistemlerin sınırlarını gerçek tehdit aktörlerinden önce aşmaya çalışır. Bilgisayar, ağ ve web teknolojilerinin (TCP/IP stack, HTTP mimarisi, işletim sistemi çalışma prensipleri) temel işleyişine hakimiyet kurulduktan sonra atılan bu adım, sahip olunan teknik bilginin "Saldırgan Bakış Açısı" (Attacker's Perspective) ile yeniden yorumlanmasını gerektirir.

Ofansif güvenlik süreci, statik kurallarla değil, analitik ve sorgulayıcı bir zihniyetle başlar. Bir sisteme yaklaşırken temel alınan sorular ve bunların arka plandaki teknik anlamları şunlardır:

- **Dışarıya açık olan nedir?** (Hedefin **Attack Surface** yani saldırı yüzeyinin haritalandırılması: Açık portlar, yayınlanan servisler, API uç noktaları.)
- **Nelere erişilebilir?** (Erişim kontrollerinin ve **Authentication/Authorization** mekanizmalarının sınırlarının test edilmesi.)
- **Sistem hangi varsayımlarda bulunuyor?** (Geliştiricinin "Kullanıcı her zaman beklenen veri tipini girer" varsayımının çürütülmesi. Bu adım, sistemin beklenmeyen veya manipüle edilmiş girdilere nasıl tepki verdiğini gözlemlemek için kritik bir metodolojidir.)

Bu metodik yaklaşım sayesinde bir sistemin çöktüğü, beklenmedik bir hata mesajı (Stack Trace vb.) döndürdüğü veya yetkisiz erişime izin verdiği anlar tespit edilir.

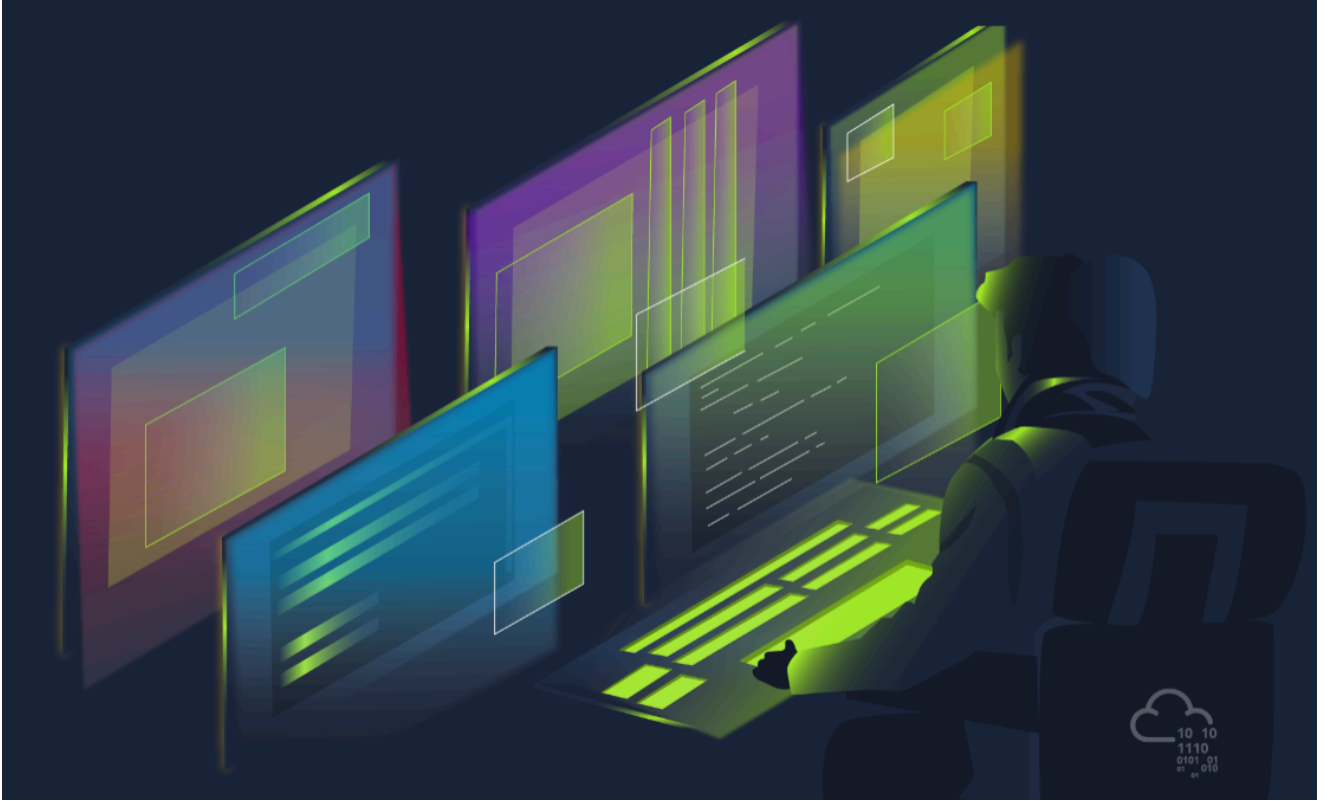
Penetration Testing (Sızma Testi) ve Metodolojik Yapı

Bu çalışma alanında "Hacking" kavramı, rastgele ve yıkıcı bir eylemden ziyade **Penetration Testing** (Sızma Testi - güvenlik zafiyetlerini tanımlamak, sömürmek ve raporlamak için kullanılan etik, yasal ve endüstri standartlarına dayalı yapılandırılmış test metodu) olarak ele alınır. Penetration Testing'in nihai amacı, sistem zayıflıklarını sömürülebilir hale gelmeden önce tespit edip iyileştirilmesini (Remediation) sağlamaktır.

ÖNEMLİ: Gerçek dünya senaryolarında (Hands-on Scenarios), modern sistemler nadiren tek bir devasa açıkla ele geçirilir. İşin asıl teknik derinliği, saldırganların hedefin daha derinlerine inmek (Örneğin, iç ağda **Lateral Movement** yapmak) için buldukları düşük seviyeli zayıflıkları ve yapılandırma hatalarını birbirine bağlamasında (**Chaining Vulnerabilities**) yatar. Başarılı bir ofansif güvenlik uzmanı, sadece tekil açıkları bulan değil, bu açıkları zincirleyerek hedefteki nihai etkiyi (Impact) maksimize eden kişidir.

Finding Weaknesses: Hedef Sistemde Zafiyet Tespiti ve Keşif

Siber güvenlikte savunma mekanizmalarının gerçek dünya koşullarında ne kadar dayanıklı olduğunu ölçmek, saldırganların (Threat Actors) hamlelerini önceden tahmin edip simüle etmekle mümkündür. Bu aşamada, hedef sistemin dış dünyaya bakan yüzünü (Attack Surface) analiz ederek, hassas verilere veya yetkili panellere açılan gizli yolları keşfetmeye odaklanıyoruz.



Ofansif Güvenlik Çekirdek Terminolojisi

Bir sisteme saldırmadan önce, gerçekleştirilen eylemin yasal ve metodolojik çerçevesini belirleyen temel kavramları içselleştirmek şarttır. Tüm bu kavramların merkezinde **Permission (İzin/Yetki)** kavramı yatar. İzin olmadan yapılan her türlü eylem yasadışıdır.

- **Red Teaming (Kırmızı Takım Egzersizi):** Belirli bir kapsam (Scope) dahilinde, gerçek bir tehdit aktörünün (Adversary) taktik, teknik ve prosedürlerini (TTPs) birebir simüle eden, kurumun tespit ve müdahale (Blue Team) kapasitesini ölçmeyi hedefleyen yapılandırılmış saldırı metodolojisidir.

- **Penetration Test (Sızma Testi):** Yetkilendirilmiş bir uzmanın, gerçek dünya risklerini anlamak amacıyla yine belirli bir kapsam içinde kalarak zafiyetleri tespit ettiği ve bunları sömürmeye (Exploitation) çalıştığı spesifik güvenlik değerlendirmesidir. Red Teaming'den farkı; genellikle gizlilik (Stealth) gerektirmemesi ve tüm zafiyetleri bulmaya odaklanmasıdır.
- **Vulnerability (Zafiyet):** Bir sistemde, uygulamada veya konfigürasyonda bulunan ve bir saldırgan tarafından kötüye kullanılabilecek (Abuse) mantıksal veya yapısal zayıflıktır.
- **Exploit (İstismar):** Tespit edilen bir Vulnerability'i kullanarak, kısıtlı bir fonksiyona veya veriye erişim gibi spesifik bir sonucu elde etmek için kullanılan teknik, kod parçası veya metodolojidir.
- **Scope (Kapsam):** Bir görevlendirme (Engagement) sırasında nelerin test edilebileceğinin, hangi sistemlerin (IP blokları, domainler), uygulamaların ve eylemlerin izinli olduğunun (veya yasaklı olduğunun - Off-limits) kesin sınırlarıdır.

Web Uygulamalarında Keşif (Reconnaissance) ve Manuel Analiz

Bir web uygulamasının güvenliğini test ederken, geliştiricilerin "kimse bu URL'yi tahmin edemez" (Security through Obscurity) yanılgısıyla açıkta bıraktığı sayfaları bulmak ilk adımlardan biridir. Bu senaryoda hedefimiz <http://www.onlineshop.thm/> adresidir.

Geliştirici panelleri, yedekleme dosyaları veya yönetim arayüzleri genellikle kök dizin (Root Directory) altında spesifik yollarda (Paths) saklanır. Manuel keşif aşamasında, tarayıcının adres çubuğuna tahmin edilen dizin adları eklenerek uygulamanın vereceği HTTP yanıt kodları (HTTP Response Codes) gözlemlenir.

ÖNEMLİ: Manuel analizde temel alınan mantık, uygulamanın var olmayan bir kaynağa karşı vereceği tepkiyi (Baseline) ölçmektir. Hedef sisteme `/sitemap`, `/mail`, `/register`, `/login`, `/admin` gibi yollar eklediğimizde arka planda şu teknik süreç işler:

Eğer girdiğimiz dizin sunucuda fiziksel veya mantıksal olarak bulunmuyorsa, sunucu standart bir **HTTP 404 Not Found** (Sayfa Bulunamadı) hatası döndürür. Bu hata, isteğin sunucuya ulaştığını ancak kaynağın mevcut olmadığını kanıtlar. Eğer sistem bir dizin için 404 yerine **HTTP 200 OK** veya yetki eksikliği belirten **HTTP 403 Forbidden** döndürüyorsa, bu dizinin sunucuda var olduğu kesinleşir ve saldırı yüzeyi genişler.

Otomatize Dizin Keşfi (Automated Directory Enumeration)

Manuel denemeler, kontrol edilecek yol sayısı az olduğunda etkilidir. Ancak on binlerce potansiyel gizli dosya veya dizini bulmak için (örneğin `.git` klasörleri, `config.php.bak` dosyaları) **Gobuster** gibi otomatize edilmiş Brute-Force (Kaba Kuvvet) araçlarına ihtiyaç duyulur.

Gobuster, Go diliyle yazılmış, çoklu iş parçacığı (Multi-threading) kullanarak hedef web sunucusuna saniyeler içinde binlerce HTTP GET isteği gönderen ve dönen HTTP durum

kodlarına göre izinleri veya dosyaları haritalayan (Directory/File Enumeration) güçlü bir komut satırı aracıdır.

Terminalimizde aşağıdaki komutu çalıştırarak keşif sürecini başlatıyoruz:

```
gobuster dir --url http://www.onlineshop.thm/ -w /usr/share/wordlists/dirbuster/directory-list.txt
```

Bu komutun teknik olarak deşifre edilmesi ve arka planda işleyen mantık şu şekildedir:

- **gobuster** : Çalıştırdığımız ana aracın (Binary) ismidir.
- **dir** : Gobuster'ın çalışma modunu (Mode) belirler. Araç DNS, Vhost veya S3 bucket araması da yapabilir ancak **dir** modülü, spesifik olarak web sunucusu üzerinde izin ve dosya keşfi (URI Enumeration) yapmak için kullanılır.
- **--url http://www.onlineshop.thm/** : Hedef (Target) adresi belirtir. İsteğin gideceği kök URL'dir. Protokolün (HTTP/HTTPS) doğru belirtilmesi zorunludur.
- **-w /usr/share/wordlists/dirbuster/directory-list.txt** : Aracın deneyeceği olası izin isimlerinin bulunduğu kelime listesinin (Wordlist) tam dosya yoludur (Absolute Path). Araç bu dosyadaki her bir satırı okur ve hedefin sonuna ekleyerek (Örn: `http://www.onlineshop.thm/admin`) istek atar. Bu dosya yolu, Kali Linux gibi ofansif dağıtımlarda standart olarak gelen SecLists/DirBuster kelime listesi lokasyonudur.

Mantıksal Analiz (Neden bu parametreleri kullanıyoruz?): Sisteme körü körüne saldırmak yerine, önceden derlenmiş ve sızdırılmış veritabanlarından veya genel geliştirici alışkanlıklarından oluşturulmuş bir kelime listesi (`directory-list.txt`) kullanıyoruz. Bu, zaman maliyetini düşürür ve isabet oranını artırır.

DİKKAT (B Planı ve Hata Yönetimi): Eğer bu **gobuster** komutu başarısız olursa veya hedef sunucudan sürekli "Timeout" ya da "HTTP 429 Too Many Requests" (Aşırı İstek / Rate Limiting) hatası alırsak, hedefin arkasında bir WAF (Web Application Firewall) veya IPS (Intrusion Prevention System) olduğu anlaşılır. Bu durumda B planı olarak; **gobuster** komutuna **-t 10** parametresi eklenerek eşzamanlı istek (Thread) sayısı düşürülür veya **--delay 2s** parametresi ile her istek arasına bekleme süresi konularak güvenlik cihazlarının tespit eşiklerinin (Detection Threshold) altında kalınmaya çalışılır. Alternatif araç olarak farklı filtreleme özelliklerine sahip **ffuf** veya **wfuzz** kullanılabilir.

Exploiting Weaknesses: Zafiyetleri İstismar Etmek ve Zincirleme Saldırı Çevrimi

Zafiyet Zincirleme (Chaining Vulnerabilities) ve Başlangıç Noktası

Siber güvenlikte, tek başına ele alındığında kritik görünmeyen veya yalnızca **Information Disclosure** (Bilgi İfşası) olarak sınıflandırılacak zafiyetler, doğru bir metodolojiyle

birleştirildiğinde tüm sistemin ele geçirilmesine yol açabilir. Zafiyet zincirleme mantığı tam olarak budur; sistemdeki güvenlik açıklarını birer domino taşı gibi konumlandırıp, birinin düşüşünün diğerlerini tetiklemesini sağlamaktır.

Önceki aşamada **Gobuster** ile tespit edilen gizli giriş sayfası (/login), savunma hattındaki ilk çatlak, yani ilk domino taşıdır. Geliştirici, bu sayfayı gizli tutarak güvende olacağını (Security through Obscurity) varsaymıştır. Ofansif güvenlik uzmanının temel prensibi olan "Sistem beklenen dışında bir duruma nasıl tepki verir?" (Adversarial Mindset / Düşmanca Zihniyet) sorusu burada devreye girer. Bu sayfanın tespiti, saldırı yüzeyini (Attack Surface) genişletmiş ve bizi doğrudan **Authentication** (Kimlik Doğrulama) mekanizmasını test etmeye yönlendirmiştir.

ÖNEMLİ: Başarılı bir sızma testi, geliştiricinin "Kullanıcı bunu asla denemez" dediği sınır durumlarını (Edge Cases) test etmek, sistemleri manipüle etmek (Fuzzing) ve küçük zayıflıkları birleştirerek kritik bir etki (Impact) yaratmak üzerine kuruludur.

Hedefin Değeri: Neden Kimlik Bilgilerini Hedefliyoruz?

Bir web uygulamasında dışarıdan (Unauthenticated) erişilebilen alanlar kısıtlıdır. Saldırganların öncelikli amacı, geçerli kimlik bilgileri (Credentials) elde ederek sistemin iç katmanlarına (Authenticated yüzeyine) inmektir. Yetkili erişim sağlandığında elde edilecek avantajlar şunlardır:

- **Hassas Fonksiyonlara Erişim (Sensitive Functionality):** Sadece yetkili kullanıcılara açık olan, veri değiştirme veya silme gibi uygulamanın iş mantığını (Business Logic) barındıran kritik fonksiyonların manipüle edilmesi.
- **Kullanıcı Verilerinin İfşası (User Data Breach):** Kredi kartı bilgileri, PII (Personally Identifiable Information - Kişisel Tanımlanabilir Bilgiler) gibi verilerin çalınması, veritabanına sızılması.
- **Yönetimsel Ayrıcalıklar (Administrative Features):** Eğer ele geçirilen hesap bir Admin hesabıysa; sistem ayarlarının değiştirilmesi, yeni zararlı kullanıcılar eklenmesi (Persistence/Kalıcılık sağlama) veya dosya yükleme (File Upload) fonksiyonlarının kötüye kullanılarak sunucuda komut çalıştırılmasına (Remote Code Execution - RCE) olanak tanınması.
- **Ağ İçi İlerleme (Lateral Movement & Pivot):** Ele geçirilen bu sistem üzerinden, kurumun iç ağındaki diğer sunuculara veya veritabanlarına sıçrama noktası (Pivot) yaratılması.

Otomatize Sözlük Saldırısı (Dictionary Attack) ve Hydra

Gizli /login sayfasında "admin" kullanıcısına yönelik birkaç yaygın parolayı manuel olarak denemek (Örn: 123456 , password) sadece zayıf konfigürasyonları tespit etmenin ilk adımındır. Ancak gerçek dünya senaryolarında, binlerce parolayı kapsayan devasa sözlük listeleriyle (Wordlist) otomatize bir saldırı düzenlemek şarttır.

Bu noktada ağ tabanlı kimlik doğrulama kırma işlemlerinde endüstri standardı olan **Hydra** devreye girer. Hydra, paralel bağlantılar (Multi-threading) kurarak farklı servislere (SSH, FTP, HTTP, SMB vb.) yönelik hızlı bir **Dictionary Attack** (Sözlük Saldırısı - önceden tanımlanmış olası parolalar listesinin hedef sisteme sırayla ve hızla gönderilmesi işlemi) gerçekleştirir.

Aşağıdaki komut, uygulamanın kimlik doğrulama mekanizmasını kırmak için yapılandırılmış spesifik bir Hydra payload'udur:

```
hydra -l admin -P passlist.txt www.onlineshop.thm http-post-form  
"/login:username=^USER^&password=^PASS^:F=incorrect" -V
```

Komutun ve Parametrelerin Mantıksal Analizi:

- **hydra** : Çalıştırılan ağ tabanlı Brute-Force aracının kendisi.
- **-l admin** : Hedeflenen spesifik kullanıcı adını (Login) belirtir. (Eğer elimizde bir kullanıcı adı listesi olsaydı küçük **-l** yerine büyük **-L users.txt** kullanırdık).
- **-P passlist.txt** : Parolaların (Passwords) bulunduğu kelime listesinin dosya yoludur. Araç, bu dosyadaki her satırı tek tek okuyup deneyecektir.
- **www.onlineshop.thm** : İsteğin gönderileceği hedefin domain veya IP adresidir.
- **http-post-form** : Hydra'nın kullanacağı modüldür. Login işlemi, verileri URL üzerinde değil HTTP gövdesinde (Body) ileten bir POST isteği olduğu için bu modül seçilmiştir.
- **"/login:username=^USER^&password=^PASS^:F=incorrect"** : En kritik konfigürasyon dizgisidir. Üç ana bileşene ayrılır ve : (iki nokta üst üste) ile bölünür:
 1. **/login** : POST isteğinin yapılacağı uygulamanın izin yolu (URI Path).
 2. **username=^USER^&password=^PASS^** : Tarayıcının gönderdiği HTTP Body formatının birebir kopyasıdır. **^USER^** ve **^PASS^** ifadeleri Hydra'nın özel değişkenleridir. Çalışma zamanında (Runtime), **-l** parametresindeki "admin" **^USER^** yerine, **passlist.txt** içindeki o anki parola ise **^PASS^** yerine dinamik olarak yerleştirilir.
 3. **F=incorrect** : Başarısızlık (Failure) koşuludur. Hydra, gönderdiği isteğe dönen HTTP yanıtının içeriğini (Response Body) inceler. Eğer dönen sayfada "incorrect" (veya hatalı şifre girdiğinizde çıkan uyarı metni neyse o) kelimesi geçiyorsa (**F=**), bu parolanın yanlış olduğunu anlar.
- **-v** : Verbose (Ayrıntılı) modu aktifleştirir. Arka planda denenen her bir kullanıcı adı/parola kombinasyonunu terminale yazdırarak sürecin takılmadan ilerlediğini canlı olarak doğrulamamızı sağlar.

DİKKAT (B Planı ve Hata Toleransı): Eğer Hydra bu komutla tüm parolaları bulduğunu iddia ediyor (False Positive) veya hiçbir şey bulamıyorsa (False Negative), sorun genellikle **F=incorrect** parametresinin yanlış tanımlanmasındandır. Yanıt sayfasındaki hata mesajı tam olarak eşleşmiyorsa araç yanılır. B Planı olarak; başarısızlık koşulu (**F=**) yerine, sadece başarılı girişte ekranda beliren "Welcome admin" gibi bir metni baz alan Başarı koşulu (**S=Welcome**) parametresi kullanılarak mantık tersine çevrilmelidir. Ayrıca, sistemde bir **Account Lockout Policy** (Belirli sayıdaki hatalı denemede hesabı kilitleme politikası) veya

CAPTCHA mekanizması varsa, Hydra engellenecektir. Bu senaryoda doğrudan Brute-Force bırakılmalı ve parola sıfırlama (Password Reset) mekanizmasındaki mantıksal zafiyetlere veya OAUTH/SSO atlatma (Bypass) tekniklerine odaklanılmalıdır.